

STAR-GS: Truthful and Visibility-Aware Resource Scheduling for Ground Station as a Service

Zhiying Wang*, Xiaojian Wang[†], Huayue Gu[‡], Zhishan Guo*, Ruozhou Yu*

*Department of Computer Science, North Carolina State University, Raleigh, NC 27606, USA

[†]Department of Computer Science and Engineering, University of Colorado Denver, Denver, CO 80204, USA

[‡]Department of Information Technology, Kennesaw State University, Kennesaw, GA 30144, USA

Abstract—The rapid growth of Low Earth Orbit satellite constellations has created increasing demand for efficient and scalable downlink services. Ground Station as a Service (GSaaS) provides an on-demand access model for satellite operators, but commercial GSaaS providers must schedule limited ground station bandwidth among multiple satellites with heterogeneous data demands, overlapping visibility windows, strict deadlines, and strategic bidding behaviors. This paper studies GSaaS resource scheduling from a ground station-centric perspective, where the provider actively decides task admission, ground station assignment, bandwidth allocation, and payment determination. Under satellite orbital dynamics, bandwidth constraints, and downlink task deadlines, it becomes NP-hard for the GSaaS provider to maximize its revenue. To address this challenge, we propose STAR-GS, a truthful and feasibility-aware scheduling mechanism that combines bid-aware admission control, best-fit ground station assignment, Earliest Deadline First (EDF)-based bandwidth scheduling, and critical payment pricing. Combining auction theory and schedulability theory, our novel mechanism ensures satellites truthfully report their internal valuation during task bidding, while any admitted subset of tasks is feasibly schedulable based on their deadlines. Simulations using Ansys Systems Tool Kit (STK) show that STAR-GS consistently achieves higher revenue than heuristic baselines and obtains near-MILP performance with substantially lower runtime, demonstrating its effectiveness for scalable GSaaS resource scheduling.

Index Terms—Ground Station as a Service, LEO satellites, resource scheduling, truthful auction, mechanism design, bandwidth allocation

I. INTRODUCTION

The rapid growth in the number of Low Earth Orbit (LEO) satellites, together with applications such as remote sensing, IoT, and meteorological observation [1]–[3], has created substantial demand for downlink data transmission. Despite that, satellite-ground downlink remains a critical bottleneck, constrained by limited satellite-ground visibility and scarce ground station availability for most LEO constellations. Except for designated broadband LEO constellations such as Starlink or Kuiper [4], building and operating dedicated ground stations is costly, and each ground station offers only limited coverage. To address the gap, commercial Ground Station as a Service (GSaaS) providers emerge [5], each operating a shared ground stations infrastructure with wide coverage, and leasing their ground station resources to satellite operators on demand.

As LEO satellites move rapidly, their downlink demands vary significantly across locations and time. This spatio-temporal imbalance in downlink demand can lead to con-

gestion at hotspot locations while leaving ground-station resources underutilized elsewhere [6], [7]. Effective satellite-ground downlink resource scheduling is crucial for improving resource utilization while satisfying the demands of diverse LEO applications. To optimize downlink resources in GSaaS, existing works adopt a satellite-centric perspective, where each satellite acts as the decision maker and selects one or multiple ground stations to minimize downlink latency, maximize throughput, or reduce operational cost [5], [8]. In this scenario, ground stations are modeled as passive resource providers with predefined capacities, while satellites compete or coordinate for the use of ground resources on their own. The model works well when the downlink load is mild and/or static [9]. However, when ground stations are shared and capacity-constrained, serving multiple satellites with heterogeneous demands and overlapped visibility windows, existing satellite-centric solutions fail to satisfy application requirements and resource utilization [10].

In this paper, we study GSaaS resource scheduling from a ground-station provider’s perspective, where the provider centrally determines task admission and resource allocation to maximize service revenue. This setting presents three key challenges. First, short-lived satellite-ground contacts and overlapping visibility windows make it difficult to complete heterogeneous tasks before their deadlines [11]. Second, private task valuations create incentives for strategic bidding, requiring joint admission and pricing decisions. Third, task admission, ground-station assignment, and bandwidth scheduling are tightly coupled, as admitting a high-value task may consume scarce visibility windows needed by other urgent tasks. These challenges lead to a non-trivial revenue-maximization problem involving deadline-aware scheduling, visibility-aware assignment, and incentive-compatible pricing.

To address these intricate challenges, this paper makes the following major contributions:

- We study GSaaS resource scheduling under dynamic satellite-ground visibility, task-specific deadlines, and provider reserve-price constraints, where the provider jointly determines task admission, ground-station assignment, bandwidth allocation, and payment. We formulate this setting as a GSaaS resource scheduling and revenue maximization problem, and prove that the problem is NP-hard.
- We propose Scheduling via Truthful Auction and

Resource-aware Ground-Station Selection (STAR-GS), a computationally efficient, feasibility-aware scheduling and truthful auction mechanism that combines bid-aware admission, best-fit ground-station assignment, Earliest Deadline First (EDF) scheduling, and critical payment pricing.

- We theoretically prove that STAR-GS ensures the completion of admitted tasks through feasibility-aware scheduling, guarantees truthful reporting of private task values, satisfies individual rationality for task owners, and enforces provider-side reserve-price protection.
- With extensive simulations, we show that STAR-GS achieves near-optimal revenue, outperforms state-of-the-art baselines, and maintains low time complexity and high scalability.

Organization. §II reviews related work on GSaaS resource scheduling and auction-based resource allocation. §III presents the system model and formulates the GSaaS revenue maximization problem. §IV introduces the STAR-GS mechanism and analyzes its theoretical properties. §V evaluates the performance of STAR-GS. §VI concludes this paper.

II. RELATED WORK

Research on GSaaS and satellite-ground networking has studied service architecture, decentralized ground-station sharing, satellite-centric ground-station selection, and learning-based downlink optimization. Existing works have considered sustainable offloading through commercial ground stations [8], blockchain-based GS federation for privacy-preserving nanosatellite communication [12], cybersecurity analysis of GSaaS architectures [13], and reinforcement-learning-based downloading under atmospheric uncertainty [14]. Recent studies further optimize multi-provider GSaaS usage and satellite-centric site selection, such as SkyGS [15] and scalable GSaaS site clustering [16]. These studies provide effective solutions for scenarios where satellites select suitable ground-station services to improve latency, reliability, cost, or coverage. In contrast, our work focuses on the complementary provider-side setting, where a GSaaS operator manages shared ground-station resources and jointly decides task admission, ground-station assignment, bandwidth scheduling, and pricing.

Resource scheduling for satellite-ground station networks has also been widely investigated. Existing studies have designed constellation-wide scheduling pipelines to mitigate ground-track congestion [17], reinforcement-learning-based hierarchical scheduling to improve utilization and profit [18], and mixed-integer or genetic-optimization models for satellite-ground network planning [19]. These approaches are well suited to improving scheduling feasibility, throughput, resource utilization, or planning efficiency when task requirements and priorities are specified by the system. Our work considers a different commercial GSaaS setting, where task owners may have private valuations and the provider needs to make admission and pricing decisions while still satisfying visibility and deadline constraints.

TABLE I
MAIN NOTATIONS

Notation	Description	Notation	Description
A_{ik}	Arrival slot	$\mathcal{A}_j(t)$	Serviceable tasks at GS j
B_j	GS bandwidth	b_{ijkt}	Allocated bandwidth
bid_{ik}	Unit bid	c_{ijk}	Processing requirement
$\mathcal{C}_{ij}$	Visibility windows	D_{ik}	Task deadline
$\mathcal{F}_{ik}$	Candidate GS set	$\mathcal{G}$	GS set
$g(i, k)$	Assigned GS	$I_j(t)$	Unused slot fraction
$\mathcal{K}_i$	Tasks of satellite i	M_{ij}	Number of windows
p_{ik}	Unit price	P_{ik}	Total payment
$\mathcal{R}$	Request set	$\mathcal{R}^+$	Reserve-eligible requests
$\mathcal{S}$	Satellite set	Slack_j	Residual slack
$\mathcal{T}$	Time-slot set	$\mathcal{T}_{ijk}$	Feasible service window
u_{ik}^{GS}	Provider utility	u_{ik}^{task}	Task-owner utility
v_{ik}	Private value	$\mathcal{W}_j$	Tasks assigned to GS j
x_{ijk}	Assignment indicator	y_{ik}	Completion indicator
α_{ijkt}	Service fraction	Δt	Slot duration
$\Delta \tau_{ijkt}$	Transmitted data	η_{ij}	Spectral efficiency
ρ	Reserve price	τ_{ik}	Data demand
$\tau_{ik}^{\text{rem}}(t)$	Remaining data	$\mathcal{W}$	Candidate/winner set

Auction mechanisms have been widely used for allocating scarce resources with private valuations. Existing studies have designed pricing and auction mechanisms for edge computing and communication-resource allocation, achieving properties such as truthfulness, budget balance, and efficient resource utilization [20], [21]. Other game-theoretic or Bayesian approaches further optimize surplus and resource efficiency in terrestrial networks. These mechanisms provide useful tools for incentive-aware resource allocation in many networked systems. Compared with these settings, GSaaS scheduling introduces additional structure from intermittent satellite-ground visibility, task-specific deadlines, and ground-station bandwidth constraints. Therefore, our work adapts the mechanism-design perspective to a visibility-aware and deadline-aware GSaaS scheduling problem.

Building on these lines of work, STAR-GS studies provider-side GSaaS resource scheduling by jointly considering revenue maximization, satellite-ground visibility, task deadlines, bandwidth feasibility, and private task valuations. Unlike satellite-centric selection, conventional scheduling, and general auction mechanisms, STAR-GS integrates provider-side admission, assignment, bandwidth scheduling, bid-aware pricing, and visibility- and deadline-aware schedulability checking into a revenue-aware, feasibility-aware, and incentive-compatible mechanism.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we study GSaaS resource scheduling from the perspective of a service provider, as shown in Fig. 1. We consider a GSaaS service provider operating a set of geo-distributed ground stations and serving a set of individual satellites that request downlink services. Each satellite submits downlink tasks to the GSaaS provider before the corresponding deadlines, and the provider actively determines task admission, ground-station assignment, task scheduling, bandwidth allocation, and payments.

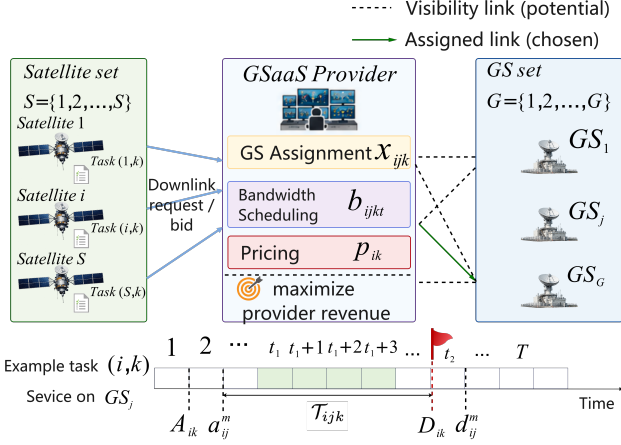


Fig. 1. System model of the GSaaS resource allocation framework. Satellites submit downlink requests and bids to the GSaaS provider, which determines task admission, GS assignment x_{ijk} , bandwidth allocation b_{ijkt} , and pricing p_{ik} . Dashed links indicate potential visibility between satellites and ground stations, while solid links indicate chosen assignments. The bottom timeline shows how the feasible service window T_{ijk} of task (i, k) is formed by intersecting its lifetime $[A_{ik}, D_{ik}]$ with the visibility window $[a_{ij}^m, d_{ij}^m]$.

A. System Entities and Time Model

Let $\mathcal{S} = \{1, 2, \dots, S\}$ denote the set of satellites that generate downlink requests within a scheduling horizon. Similarly, let $\mathcal{G} = \{1, 2, \dots, G\}$ denote the set of commercial Ground Stations (GSs) operated by the service provider.

Following existing work [22], we assume that time is discretized into a set of equal-duration slots, indexed by $\mathcal{T} = \{1, 2, \dots, T\}$, where each slot has duration Δt . This common discretization maps heterogeneous visibility windows onto a finite scheduling horizon and can be extended to irregular time slots by using slot-dependent durations.

B. Satellite Downlink Requests

Each downlink request is modeled as a persistent communication task characterized by its arrival time, deadline, and data demand:

$$(A_{ik}, D_{ik}, \tau_{ik}), \quad i \in \mathcal{S}, k \in \mathcal{K}_i. \quad (1)$$

Here, A_{ik} denotes the arrival time slot of the k -th downlink task generated by satellite i , D_{ik} denotes the deadline by which the task must be completed, and τ_{ik} denotes the amount of data to be downlinked. **We assume that τ_{ik} is declared by the task owner when the request is submitted. This assumption is suitable for buffered downlink tasks, such as stored imagery, telemetry, and mission files, whose data volume is known before scheduling.**

Each downlink task can be requested by a different application-level user through satellite tasking platforms such as Planet or SkyFi, and thus may have its own private valuation [23], [24]. We therefore treat each task owner as an independent auction participant and define value, utility, and payment at the task level; a task is paid only if its cumulative downlink reaches τ_{ik} by deadline D_{ik} .

C. Commercial Ground Stations and Visibility

We consider a multi-GS setting in which a GSaaS provider operates a set of geographically distributed ground stations $\mathcal{G}$. Each ground station $j \in \mathcal{G}$ is equipped with available downlink bandwidth B_j . A ground station can serve multiple satellites in the same time slot through Frequency-Division Multiple Access (FDMA) [25]. We consider a finite planning horizon consisting of the time slots in $\mathcal{T}$. Due to orbital dynamics and line-of-sight constraints, a satellite can communicate with a ground station only during specific contact intervals within this planning horizon. Since a satellite may have multiple contacts with the same ground station during the horizon, we denote the set of visibility windows $\mathcal{C}_{ij}$ between satellite i and ground station j as

$$\mathcal{C}_{ij} = \{[a_{ij}^m, d_{ij}^m]\}_{m=1}^{M_{ij}}, \quad (2)$$

where M_{ij} is the number of visibility windows between satellite i and ground station j , and $[a_{ij}^m, d_{ij}^m]$ denotes the m -th visibility window. **These visibility windows are deterministically derived from the predicted satellite orbit and the geographic location of the ground station using line-of-sight geometry.**

For each downlink task (i, k) and ground station j , we define its feasible service window as the set of time slots that are both within the task lifetime and covered by at least one visibility window:

$$\mathcal{T}_{ijk} \triangleq \left(\bigcup_{m=1}^{M_{ij}} [a_{ij}^m, d_{ij}^m] \right) \cap [A_{ik}, D_{ik}]. \quad (3)$$

D. Bandwidth Allocation and Ground Station Assignment

We consider two scheduling decisions made by the GSaaS provider: ground station assignment and bandwidth allocation. Let $b_{ijkt} \geq 0$ denote the bandwidth allocated by GS j to downlink task (i, k) at time slot t . The ground station assignment variable is defined as

$$x_{ijk} \in \{0, 1\}, \quad \forall i \in \mathcal{S}, k \in \mathcal{K}_i, j \in \mathcal{G}, \quad (4)$$

where $x_{ijk} = 1$ indicates that downlink task (i, k) is assigned to GS j , otherwise $x_{ijk} = 0$. The bandwidth allocation and ground station assignment decisions are subject to the following physical and scheduling constraints.

At each time slot, the aggregate bandwidth allocated by each GS cannot exceed its available bandwidth:

$$\sum_{i \in \mathcal{S}} \sum_{k \in \mathcal{K}_i} b_{ijkt} \leq B_j, \quad \forall j \in \mathcal{G}, t \in \mathcal{T}. \quad (5)$$

A task can receive bandwidth from a GS only if it is assigned to that GS. This coupling between bandwidth allocation and GS assignment is enforced by

$$0 \leq b_{ijkt} \leq x_{ijk} B_j, \quad \forall i \in \mathcal{S}, k \in \mathcal{K}_i, j \in \mathcal{G}, t \in \mathcal{T}. \quad (6)$$

In addition, a GS can serve a task only when the satellite is visible to the GS and the task has arrived but not yet expired.

Therefore, bandwidth allocation is restricted to the feasible service window $\mathcal{T}_{ijk}$:

$$b_{ijkt} = 0, \quad \forall t \notin \mathcal{T}_{ijk}, \quad \forall i \in \mathcal{S}, k \in \mathcal{K}_i, j \in \mathcal{G}. \quad (7)$$

Following practical GSaaS contact-reservation operations [26], we model each downlink task as being explicitly reserved at a specific ground station. This does not restrict a satellite to a single ground station: different tasks from the same satellite may be assigned to different ground stations across different contact opportunities. Therefore, each downlink task can be assigned to at most one primary GS, which is captured by

$$\sum_{j \in \mathcal{G}} x_{ijk} \leq 1, \quad \forall i \in \mathcal{S}, k \in \mathcal{K}_i. \quad (8)$$

E. Request Completion

Let η_{ij} denote the spectral efficiency (bps/Hz) of the downlink between satellite i and ground station j . **We model η_{ij} as a fixed effective spectral efficiency for each satellite-GS pair, representing an average link rate over the scheduling horizon.** The amount of data transmitted for downlink task (i, k) from satellite i to GS j during time slot t is given by

$$\Delta\tau_{ijkt} = b_{ijkt} \cdot \eta_{ij} \cdot \Delta t, \quad \forall i \in \mathcal{S}, k \in \mathcal{K}_i, j \in \mathcal{G}, t \in \mathcal{T}. \quad (9)$$

Since a downlink task may span multiple time slots, we track its remaining data volume over time. Let $\tau_{ik}^{\text{rem}}(t)$ denote the remaining amount of data of task (i, k) at the beginning of time slot t . Its evolution is given by

$$\tau_{ik}^{\text{rem}}(t+1) = \max \left\{ 0, \tau_{ik}^{\text{rem}}(t) - \sum_{j \in \mathcal{G}} \Delta\tau_{ijkt} \right\}, \quad (10)$$

$\forall i \in \mathcal{S}, k \in \mathcal{K}_i.$

The completion indicator is defined according to whether the remaining data volume reaches zero before the deadline:

$$y_{ik} = \begin{cases} 1, & \text{if } \tau_{ik}^{\text{rem}}(D_{ik} + 1) = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

If a downlink task cannot be completed by its deadline, it will be dropped and yield no revenue to the service provider.

F. Pricing, Utilities, and Economic Constraints

We model GSaaS pricing as an auction, where the GSaaS provider acts as the seller/auctioneer and downlink task owners act as buyers competing for limited downlink resources. Each task owner submits one downlink request and a unit bid bid_{ik} for completing this request. Following the standard single-parameter private-value setting [27], each task (i, k) has a private unit value v_{ik} , which represents the task owner's valuation for successfully completing the task. This value is private information of the task owner and is not directly observable by the GSaaS provider. As a result, a task owner may strategically misreport its willingness to pay through bid_{ik} , either by underbidding to reduce its potential payment or by overbidding to increase its chance of being admitted and scheduled.

For each admitted task (i, k) , the GSaaS provider charges a unit service price p_{ik} . Since the task requires the transmission of τ_{ik} units of data, the total payment charged to the owner of task (i, k) is $P_{ik} = p_{ik}\tau_{ik}y_{ik}$. If the task is not completed before its deadline, i.e., $y_{ik} = 0$, no payment is collected.

The utility of the owner of task (i, k) is defined as

$$u_{ik}^{\text{task}} = \begin{cases} (v_{ik} - p_{ik})\tau_{ik}, & \text{if } y_{ik} = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

The GSaaS provider uses a reserve unit price to capture its minimum acceptable payment for serving downlink traffic. In this paper, we use a uniform reserve price ρ for all tasks as a simplifying model, representing the provider's operating expense or opportunity cost per unit downlink data. This model can be extended to task-dependent or GS-dependent reserve prices when heterogeneous operating costs are considered. Under the proposed mechanism, a task is eligible for admission only if its bid satisfies $\text{bid}_{ik} \geq \rho$.

The utility of the GSaaS provider obtained from serving task (i, k) is defined as

$$u_{ik}^{\text{GS}} = \begin{cases} (p_{ik} - \rho)\tau_{ik}, & \text{if } y_{ik} = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

Thus, for any admitted and completed task, the proposed mechanism enforces the transaction price to satisfy

$$\rho \leq p_{ik} \leq \text{bid}_{ik}, \quad \forall (i, k) \text{ with } y_{ik} = 1. \quad (14)$$

This constraint ensures that each completed winning task is charged at least the provider's reserve price and no more than its reported bid.

We evaluate the mechanism by individual rationality, truthfulness, and reserve-price protection. Individual rationality requires truthful task owners to obtain non-negative utility; truthfulness, or dominant-strategy incentive compatibility, requires truthful bidding to maximize each task owner's utility regardless of others' bids; and reserve-price protection requires every completed winning task to be charged at least ρ .

G. Optimization Objective

Based on the pricing and utility model defined above, the GSaaS provider aims to maximize its total revenue. The revenue maximization problem can be formulated as

$$\begin{aligned} & \max_{\{b_{ijkt}, x_{ijk}, p_{ik}\}} \sum_{i \in \mathcal{S}} \sum_{k \in \mathcal{K}_i} p_{ik}\tau_{ik}y_{ik} \\ & \text{s.t. Constraints (4) - (11), (14).} \end{aligned} \quad (15)$$

In Problem (15), the GSaaS provider jointly optimizes bandwidth allocation, ground-station assignment, and payment determination to maximize total revenue, subject to assignment, bandwidth-capacity, visibility-window, task-completion, and pricing constraints.

Theorem 1. *Problem (15) is NP-hard.*

Proof. We reduce the 0-1 knapsack problem to Problem (15). Consider an arbitrary knapsack instance with item set $\mathcal{N} =$

$\{1, \dots, N\}$, item weights $\{w_n\}$, item values $\{q_n\}$, and capacity C . We construct a GSaaS instance with one ground station and one time slot. The ground-station bandwidth is set to $B_1 = C$, and we set $\Delta t = 1$ and $\eta_{n1} = 1$ for all satellites n .

For each item $n \in \mathcal{N}$, we create one satellite $i = n$ with a single downlink task $k = 1$, i.e., $\mathcal{S} = \{1, \dots, N\}$ and $\mathcal{K}_n = \{1\}$. The task $(n, 1)$ has demand $\tau_{n1} = w_n$, arrival time $A_{n1} = 1$, deadline $D_{n1} = 1$, feasible service window $\mathcal{T}_{n,1,1} = \{1\}$, unit bid $\text{bid}_{n1} = q_n/w_n$, and reserve price $\rho = 0$.

In this constructed instance, completing task $(n, 1)$ requires exactly w_n units of bandwidth in the only slot, while the total available bandwidth is C . Thus, any feasible set of completed tasks corresponds exactly to a feasible knapsack selection. Moreover, by setting $p_{n1} = \text{bid}_{n1}$, the revenue obtained from task $(n, 1)$ is

$$p_{n1}\tau_{n1} = \text{bid}_{n1}\tau_{n1} = \frac{q_n}{w_n}w_n = q_n. \quad (16)$$

Therefore, maximizing the GSaaS revenue is equivalent to maximizing the total value of selected knapsack items. Since 0–1 knapsack is NP-hard, Problem (15) is NP-hard. $\square$

IV. STAR-GS DESIGN

Beyond its NP-hardness, Problem (15) tightly couples task admission, ground-station assignment, bandwidth allocation, and payment determination under visibility and deadline constraints. STAR-GS addresses this coupling through bid-aware admission, best-fit ground-station assignment, EDF-based schedulability testing, and critical-payment pricing. **Intuitively, bid-aware admission prioritizes valuable tasks, EDF prevents infeasible admissions, best-fit assignment preserves flexible GS resources, and critical payments ensure truthful bidding.** For feasibility checking, STAR-GS converts fractional bandwidth allocation into equivalent preemptive full-bandwidth time-sharing under a fixed GS assignment, thereby reducing bandwidth feasibility to deadline schedulability.

A. From Bandwidth Allocation to Deadline Scheduling

The optimization problem in (15) is formulated through the bandwidth allocation variables b_{ijkt} . For a fixed GS assignment, however, downlink feasibility can be viewed as a real-time scheduling problem: a GS acts as a processor, its bandwidth as processor capacity, a downlink task as a real-time job, the task demand as the processing requirement, and the feasible service window $\mathcal{T}_{ijk}$ as the job availability interval.

Consider a fixed task (i, k) assigned to GS j . If GS j allocates its full bandwidth B_j to this task for one slot, the transmitted data is $B_j\eta_{ij}\Delta t$. Thus, the normalized processing requirement of task (i, k) on GS j , measured in full-bandwidth slots, is

$$c_{ijk} = \frac{\tau_{ik}}{B_j\eta_{ij}\Delta t}. \quad (17)$$

The task is feasible on GS j only if it receives at least c_{ijk} full-bandwidth slot equivalents within its feasible service window $\mathcal{T}_{ijk}$ in (3).

This scheduling view avoids explicitly searching over all fractional bandwidth allocations: if GS j allocates bandwidth b_{ijkt} to task (i, k) in slot t , this is equivalent to serving the task with full bandwidth B_j for an $\alpha_{ijkt} = b_{ijkt}/B_j$ fraction of the slot. By the per-slot bandwidth capacity constraint in (5), we have

$$\sum_{i \in \mathcal{S}} \sum_{k \in \mathcal{K}_i} \alpha_{ijkt} \leq 1. \quad (18)$$

Therefore, any feasible fractional allocation within a slot can be serialized into a full-bandwidth time-sharing schedule without changing the transmitted data. Conversely, any full-bandwidth time-sharing schedule induces a feasible bandwidth allocation by setting the average bandwidth in each slot. Hence, bandwidth allocation and preemptive full-bandwidth scheduling are equivalent for feasibility analysis.

Lemma 1 (Bandwidth–Scheduling Equivalence). *For a fixed GS j and a fixed assigned task set $\mathcal{W}_j$, there exists a feasible bandwidth allocation that completes every task in $\mathcal{W}_j$ before its deadline if and only if there exists a feasible preemptive full-bandwidth time-sharing schedule that completes the same task set before the same deadlines.*

Proof. We prove both directions.

First, suppose there exists a feasible bandwidth allocation $\{b_{ijkt}\}$ for GS j . In each slot t , define $\alpha_{ijkt} = b_{ijkt}/B_j$. By the bandwidth-capacity constraint, $\sum_{(i,k) \in \mathcal{W}_j} \alpha_{ijkt} \leq 1$. We can therefore replace the fractional bandwidth allocation in slot t by a full-bandwidth service interval of length $\alpha_{ijkt}\Delta t$ for each task (i, k) . This transformation preserves the transmitted data, since

$$B_j\eta_{ij}\alpha_{ijkt}\Delta t = b_{ijkt}\eta_{ij}\Delta t. \quad (19)$$

It is performed within the same slot, so visibility and deadline constraints are preserved.

Conversely, suppose there exists a feasible preemptive full-bandwidth time-sharing schedule. If task (i, k) is served by GS j for a fraction α_{ijkt} of slot t , we construct the corresponding bandwidth allocation as $b_{ijkt} = \alpha_{ijkt}B_j$. Since the time-sharing schedule uses at most one full-bandwidth server in each slot, $\sum_{(i,k)} \alpha_{ijkt} \leq 1$ and thus $\sum_{(i,k)} b_{ijkt} \leq B_j$. The transmitted data in slot t is $b_{ijkt}\eta_{ij}\Delta t$, matching the service provided by the schedule. Therefore, the constructed allocation satisfies the capacity, visibility, and completion constraints.

Therefore, feasible bandwidth allocation and feasible preemptive full-bandwidth scheduling are equivalent for a fixed GS assignment. $\square$

Lemma 1 reduces the feasibility subproblem to real-time schedulability analysis. Once tasks are assigned to a GS, checking their completion becomes a single-processor deadline scheduling problem. We therefore adopt EDF, a classical preemptive scheduling rule that is optimal for independent jobs with release times and deadlines [28], [29]. EDF is computationally lightweight and provides a deterministic schedulability test, making it well-suited for online GSaaS admission decisions under hard deadlines.

B. EDF-Based Schedulability Test

Algorithm 1 presents EDF_CHECK, an EDF-based schedulability oracle. Given a GS j and a candidate task set W_j , EDF_CHECK determines whether all tasks in W_j can be completed by their deadlines using GS j . It also returns the induced bandwidth allocation and the residual slack of the GS.

At each slot t , EDF_CHECK first identifies the set of currently serviceable unfinished tasks:

$$\mathcal{A}_j(t) = \{(i, k) \in W_j : t \in \mathcal{T}_{ijk}, \tau_{ik}^{\text{rem}}(t) > 0\}. \quad (20)$$

If $\mathcal{A}_j(t) \neq \emptyset$, EDF selects the task with the earliest deadline:

$$(i^*, k^*) = \arg \min_{(i, k) \in \mathcal{A}_j(t)} D_{ik}. \quad (21)$$

Ties are resolved by a fixed task-ID rule, which is independent of bids. GS j then serves the selected task using full bandwidth until the task is completed, the slot is exhausted, or the task becomes no longer serviceable. If the selected task completes before the end of the slot, the remaining capacity in that slot can be used to serve the next earliest-deadline task. This implements the full-bandwidth time-sharing interpretation.

The EDF schedule is accepted if every task completes before its deadline:

$$\tau_{ik}^{\text{rem}}(D_{ik} + 1) = 0, \quad \forall (i, k) \in W_j. \quad (22)$$

Otherwise, EDF_CHECK returns false. The residual slack of GS j is defined as the total unused full-bandwidth slot equivalent:

$$\text{Slack}_j = \sum_{t \in \mathcal{T}} I_j(t), \quad (23)$$

where $I_j(t)$ denotes the unused fraction of slot t after EDF scheduling. A smaller residual slack means that the task set fits more tightly on that GS, leaving other GSs less fragmented and more available for future tasks.

Lemma 2 (Tightness of EDF-Based Schedulability Test). *For a fixed GS j and a fixed assigned task set W_j , under the preemptive full-bandwidth scheduling model, EDF_CHECK(W_j, j) returns true if and only if there exists a feasible schedule that completes all tasks in W_j before their deadlines.*

Proof. If EDF_CHECK(W_j, j) returns true, it explicitly constructs a schedule that serves each task only within its feasible service window and completes all tasks by their deadlines; hence, the resulting schedule is feasible.

Conversely, suppose that a feasible schedule exists for W_j . By Lemma 1, the problem is equivalent to preemptive single-processor scheduling, with GS j as the processor and each task as an independent job with a release time, deadline, and processing requirement. Since EDF is feasibility-optimal for this scheduling model [28], [29], EDF also completes all tasks before their deadlines. Therefore, EDF_CHECK(W_j, j) returns true. $\square$

Algorithm 1: EDF_CHECK: EDF-Based Schedulability Test

Input: Assigned task set W_j on GS j ; feasible service windows $\{\mathcal{T}_{ijk}\}_{(i,k) \in W_j}$; GS bandwidth B_j

Output: Feasibility indicator Flag; bandwidth allocation $\{b_{ijk}\}$; residual slack Slack_j

- 1 Initialize $\tau_{ik}^{\text{rem}} \leftarrow \tau_{ik}$, $b_{ijk} \leftarrow 0$, and $I_j(t) \leftarrow 0$, $\forall (i, k) \in W_j$, $t \in \mathcal{T}$;
- 2 **for each** slot $t \in \mathcal{T}$ **do**
- 3 $C \leftarrow 1$; // Remaining full-bandwidth fraction
- 4 **while** $C > 0$ **do**
- 5 Form $\mathcal{A}_j(t)$ according to Eq. (20);
- 6 **if** $\mathcal{A}_j(t) = \emptyset$ **then**
- 7 $I_j(t) \leftarrow C$;
- 8 **break**;
- 9 Select (i^*, k^*) according to Eq. (21), with ties resolved by task IDs;
- 10 Serve (i^*, k^*) until it completes or C is exhausted; update b , τ^{rem} , and C according to Eqs. (9) and (10);
- 11 Compute Slack_j according to Eq. (23);
- 12 **if** Eq. (11) is not satisfied for some $(i, k) \in W_j$ **then**
- 13 **return** false, $\emptyset$, $\emptyset$;
- 14 **return** true, b_{ijk} , Slack_j ;

C. Feasibility-Aware Primary-GS Assignment

Algorithm 2 presents FEASIBLE_ASSIGN, which constructs a feasibility-aware primary-GS assignment. The EDF-based test above determines whether a given set of tasks can be scheduled on a fixed GS. We next describe how STAR-GS assigns tasks to GSs. Following the primary-GS constraint in (8), each admitted task is assigned to at most one GS. This design matches primary contact reservation and avoids cross-GS state synchronization and coordination overhead. **We acknowledge that multi-GS task splitting could improve schedulability by combining disjoint contact opportunities; supporting it would require redesigning the assignment, scheduling, and payment mechanisms and is left for future work.**

Given a candidate task set W , FEASIBLE_ASSIGN attempts to construct a feasible primary-GS assignment and the corresponding EDF schedules. It processes tasks in non-increasing order of reported bids:

$$\pi(W) = ((i_1, k_1), (i_2, k_2), \dots, (i_{|W|}, k_{|W|})), \quad (24)$$

where

$$\text{bid}_{i_1 k_1} \geq \text{bid}_{i_2 k_2} \geq \dots \geq \text{bid}_{i_{|W|} k_{|W|}}. \quad (25)$$

The use of bid order is revenue-aware: tasks with higher reported values are considered earlier, so scarce GS resources are first reserved for tasks that contribute more to the provider's potential revenue. Other fixed priority rules, such as earliest-deadline-first admission or throughput-first admission, can also

Algorithm 2: FEASIBLE_ASSIGN: Feasibility-Aware Primary-GS Assignment

Input: Candidate task set W ; GS set $\mathcal{G}$; feasible service windows $\{\mathcal{T}_{ijk}\}$; task bids $\{\text{bid}_{ik}\}$

Output: Feasibility indicator flag; GS assignment $\{x_{ijk}\}$; bandwidth allocation $\{b_{ijkt}\}$; residual slack $\{\text{Slack}_j\}_{j \in \mathcal{G}}$

- 1 Initialize $W_j \leftarrow \emptyset$, $\forall j \in \mathcal{G}$, and $x_{ijk} \leftarrow 0$, $b_{ijkt} \leftarrow 0$;
- 2 Sort W according to Eq. (25) to obtain $\pi(W)$;
- 3 **for each** $(i, k) \in \pi(W)$ **do**
- 4 Construct $\mathcal{F}_{ik}$ according to Eq. (26), retaining the tentative Slack_j returned by EDF_CHECK;
- 5 **if** $\mathcal{F}_{ik} = \emptyset$ **then**
- 6 **return** false, $\emptyset$, $\emptyset$, $\emptyset$;
- 7 Select $g(i, k)$ according to Eq. (27), with ties resolved by the smallest GS ID;
- 8 $W_{g(i, k)} \leftarrow W_{g(i, k)} \cup \{(i, k)\}$;
- 9 $x_{i, g(i, k), k} \leftarrow 1$;
- 10 **for each** $j \in \mathcal{G}$ **do**
- 11 (flag, b_{ijkt} , Slack_j) $\leftarrow$ EDF_CHECK(W_j, j);
- 12 **if** flag = false **then**
- 13 **return** false, $\emptyset$, $\emptyset$, $\emptyset$;
- 14 **return** true, x_{ijk} , b_{ijkt} , $\{\text{Slack}_j\}_{j \in \mathcal{G}}$;

be used as heuristics, but they do not directly align the admission order with the revenue objective. Importantly, ties are resolved by a fixed bid-independent rule, which is necessary for deterministic monotonicity in the auction mechanism.

Let W_j denote the set of tasks already assigned to GS j . When considering task (i, k) , STAR-GS tentatively adds it to each candidate GS j and checks whether $W_j \cup \{(i, k)\}$ remains schedulable using EDF_CHECK. The acceptable candidate set is defined as

$$\mathcal{F}_{ik} = \{j \in \mathcal{G} : \mathcal{T}_{ijk} \neq \emptyset, \text{EDF_CHECK}(W_j \cup \{(i, k)\}, j) = \text{true}\}. \quad (26)$$

If $\mathcal{F}_{ik} = \emptyset$, the candidate task set W is declared infeasible. Otherwise, STAR-GS selects a GS according to a best-fit rule:

$$g(i, k) = \arg \min_{j \in \mathcal{F}_{ik}} \widehat{\text{Slack}}_j, \quad (27)$$

where $\widehat{\text{Slack}}_j$ is the residual slack returned by EDF_CHECK after tentatively adding (i, k) to GS j . This best-fit rule is inspired by classical bin-packing heuristics [30]: the task is placed on the feasible GS where it fits most tightly, so that less-loaded GSs remain available for later tasks with more restrictive visibility windows.

D. Task Admission and Critical Payment Pricing

Algorithm 3 presents the task admission and critical-payment pricing procedure of STAR-GS. Each task (i, k) has a private unit value v_{ik} and submits a unit bid bid_{ik} . The GSaaS provider also has a reserve unit price ρ , which represents the

Algorithm 3: Truthful Admission and Critical Payment Pricing

Input: Task set $\mathcal{R}$; task bids $\{\text{bid}_{ik}\}$; reserve price ρ

Output: Winner set $\mathcal{W}$; critical payments $\{p_{ik}\}$; GS assignment x ; bandwidth allocation b

- 1 Initialize $\mathcal{W} \leftarrow \emptyset$ and $p_{ik} \leftarrow 0$, $\forall (i, k) \in \mathcal{R}$;
// Winner-selection phase
- 2 Form $\mathcal{R}^+$ according to Eq. (28);
- 3 Sort $\mathcal{R}^+$ according to Eq. (25) to obtain $\pi(\mathcal{R}^+)$;
- 4 **for each** $(i, k) \in \pi(\mathcal{R}^+)$ **do**
- 5 $\mathcal{W}^{\text{tmp}} \leftarrow \mathcal{W} \cup \{(i, k)\}$;
- 6 (flag, x , b , Slack) $\leftarrow$ FEASIBLE_ASSIGN($\mathcal{W}^{\text{tmp}}$);
- 7 **if** flag = true **then**
- 8 $\mathcal{W} \leftarrow \mathcal{W}^{\text{tmp}}$;
- 9 (flag, x , b , Slack) $\leftarrow$ FEASIBLE_ASSIGN($\mathcal{W}$);
// Critical-payment phase
- 10 **for each** $(i, k) \in \mathcal{W}$ **do**
- 11 Build the candidate bid list within $[\rho, \text{bid}_{ik}]$:
- 12 $\mathcal{B}_{ik} \leftarrow \{\rho\} \cup \{\text{bid}_{ik}\} \cup \{\text{bid}_{i'k'} : (i', k') \in \mathcal{R}, (i', k') \neq (i, k), \rho \leq \text{bid}_{i'k'} \leq \text{bid}_{ik}\}$;
- 13 Sort $\mathcal{B}_{ik}$ in non-decreasing order as $(\beta_1, \beta_2, \dots, \beta_{|\mathcal{B}_{ik}|})$;
- 14 $\ell \leftarrow 1$, $r \leftarrow |\mathcal{B}_{ik}|$;
- 15 **while** $\ell < r$ **do**
- 16 $m \leftarrow \lfloor (\ell + r)/2 \rfloor$;
- 17 Replace bid_{ik} by β_m , keeping all other bids unchanged;
- 18 Rerun the winner-selection phase to obtain $\mathcal{W}^{(\beta_m)}$;
- 19 **if** $(i, k) \in \mathcal{W}^{(\beta_m)}$ **then**
- 20 $r \leftarrow m$;
- 21 **else**
- 22 $\ell \leftarrow m + 1$;
- 23 $p_{ik} \leftarrow \beta_\ell$;
- 24 **return** $\mathcal{W}$, $\{p_{ik}\}_{(i, k) \in \mathcal{W}}$, x , b ;

minimum acceptable price for serving the task. STAR-GS first removes all tasks whose bids are below their reserve prices:

$$\mathcal{R}^+ = \{(i, k) \in \mathcal{R} : \text{bid}_{ik} \geq \rho\}. \quad (28)$$

The remaining tasks are processed in non-increasing bid order. For each task, STAR-GS tentatively inserts it into the current winner set and calls FEASIBLE_ASSIGN. The task is admitted only if the enlarged winner set remains feasible.

After the winner set is determined, STAR-GS computes a critical payment for each winning task. The critical payment is the smallest bid at which the task would still win when all other bids are fixed. In implementation, STAR-GS constructs an ordered candidate bid list within this interval, consisting of the reserve price ρ , the winner's submitted bid bid_{ik} , and other submitted bids in $[\rho, \text{bid}_{ik}]$, and then performs binary search

over this finite list. This threshold-payment rule is essential for **truthfulness** in single-parameter mechanisms.

E. Theoretical Properties

We analyze STAR-GS under the single-parameter private-value model in auction theory [31], where each task (i, k) has a private value v_{ik} and submits a bid bid_{ik} , while task demand, deadlines, visibility windows, GS bandwidths, and channel rates are public or verifiable. **Truthfulness** means that reporting $\text{bid}_{ik} = v_{ik}$ is a dominant strategy for every task.

Lemma 3 (Deletion Monotonicity of EDF_CHECK). *For a fixed GS j , if $\text{EDF_CHECK}(V, j) = \text{true}$, then for any subset $U \subseteq V$, $\text{EDF_CHECK}(U, j) = \text{true}$.*

Proof. Removing tasks does not reduce the bandwidth of GS j , shorten the feasible service window of any remaining task, or increase its demand. Hence, any subset $U \subseteq V$ is no harder to schedule than V . Since V is EDF-feasible, EDF_CHECK also completes all tasks in U by their deadlines, and therefore $\text{EDF_CHECK}(U, j) = \text{true}$. $\square$

Lemma 4 (Monotonicity of Winner Selection). *The winner-selection rule of STAR-GS is monotone in each task's bid. Fixing all other bids, for any task $e = (i, k)$, the following two properties hold: (i) if e wins with bid bid_{ik} , then it also wins with any higher bid $\text{bid}'_{ik} > \text{bid}_{ik}$; (ii) if e loses with bid bid_{ik} , then it also loses with any lower bid $\text{bid}'_{ik} < \text{bid}_{ik}$.*

Proof. Fix all bids except that of task $e = (i, k)$. We first prove the winner side. Suppose e wins with bid bid_{ik} . If its bid is increased to $\text{bid}'_{ik} > \text{bid}_{ik}$, then e can only move earlier in the non-increasing bid order, while the relative order among all other tasks remains unchanged. When e is considered under the higher bid, the set of tasks admitted before e is no larger than the corresponding set under the lower bid. Since e was feasible together with the previously admitted tasks under the lower bid, removing some previously admitted tasks cannot destroy feasibility by Lemma 3. Hence, e is still admitted under the higher bid.

We next prove the loser side. Suppose e loses with bid bid_{ik} , and consider any lower bid $\text{bid}'_{ik} < \text{bid}_{ik}$. If e were to win under the lower bid, then by the winner-side monotonicity just proved, increasing its bid from bid'_{ik} back to bid_{ik} would imply that e also wins with bid bid_{ik} . This contradicts the assumption that e loses with bid bid_{ik} . Therefore, e must also lose under any lower bid. $\square$

Lemma 5 (Critical Price as Threshold Payment). *For any winning task (i, k) produced by STAR-GS, the payment p_{ik} is its critical price. That is, fixing all other bids, task (i, k) wins if and only if $\text{bid}_{ik} \geq p_{ik}$.*

Proof. By Lemma 4, the winner-selection rule is monotone in each task's bid. Therefore, by the characterization of single-parameter mechanisms [32], there exists a critical threshold price for each task. Since STAR-GS searches the ordered finite bid list by bisection and returns the smallest bid at which the

task remains a winner, the returned payment p_{ik} is exactly this critical price. $\square$

Theorem 2. *Under the single-parameter private-value setting, STAR-GS is dominant-strategy incentive compatible, individually rational for task owners, satisfies the provider-side reserve-price constraint, and runs in polynomial time.*

Proof. We first prove the provider-side reserve-price constraint. STAR-GS removes all tasks with $\text{bid}_{ik} < \rho$ before admission. For every winning task, the critical payment satisfies $p_{ik} \geq \rho$. Therefore, every admitted and completed task is charged no less than the provider's reserve unit price:

$$p_{ik} \geq \rho, \quad \forall (i, k) \text{ with } y_{ik} = 1. \quad (29)$$

Equivalently, the provider-side utility for each admitted and completed task satisfies $u_{ik}^{GS} = (p_{ik} - \rho)\tau_{ik} \geq 0$. Thus, STAR-GS enforces provider-side reserve-price protection.

We next prove individual rationality. Under truthful bidding, $\text{bid}_{ik} = v_{ik}$. For every winning task, the critical payment is no larger than its submitted bid, i.e., $p_{ik} \leq \text{bid}_{ik} = v_{ik}$. Therefore, $u_{ik}^{task} = (v_{ik} - p_{ik})\tau_{ik} \geq 0$. Losing tasks receive no service and make no payment, so their utility is zero. Hence, STAR-GS is individually rational for task owners.

We then prove dominant-strategy incentive compatibility. By Lemma 4, the winner-selection rule is monotone in each task's bid. By Lemma 5, every winning task is charged its critical payment. By the characterization of single-parameter mechanisms [32], a monotone allocation rule combined with critical payments is dominant-strategy incentive compatible. Therefore, truthful bidding is a dominant strategy for every task owner.

Finally, STAR-GS runs in polynomial time because it performs polynomially many EDF-based schedulability checks, feasibility-aware assignments, and critical-payment searches over a finite bid set. Therefore, all stated properties hold. $\square$

V. PERFORMANCE EVALUATION

A. Simulation Setup

We evaluate STAR-GS using a discrete-time simulator driven by satellite-ground station access windows generated by Ansys Systems Tool Kit (STK), an industry-standard tool for high-fidelity LEO satellite mission simulation and access analysis [33]. We consider a Walker-style LEO constellation and a geographically distributed GSaaS network, with the default simulation parameters summarized in Table II. Each experiment varies one parameter at a time and reports the average of 10 runs with different random seeds.

B. Baselines

We compare STAR-GS with the following baselines:

- **Time-Space-Network Mixed-Integer Linear Programming (TSN-MILP).** TSN-MILP is adapted from the time-space-network formulation in Lee et al. [38]. We implement it with the same bid-price revenue maximization objective as Problem (15), i.e., maximizing $\sum_{(i,k)} \text{bid}_{ik}\tau_{ik}y_{ik}$ under the same scheduling constraints.

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Constellation type [34]	Walker-style LEO constellation
Orbital planes [34]	3
Satellites per plane [34]	8
Total satellites [34]	24
Number of ground stations [9]	6
Planning horizon [35]	24 hours
Slot duration [35]	60 s
Number of slots [35]	1440
Ground station bandwidth [36]	80 Mbps
Task demand [15]	[5, 20] GB
Unit bid [37]	[50, 100]
Deadline slack [36]	[30, 90] slots
Default number of tasks	300

It serves as an optimization benchmark for small-scale experiments.

- **Communication-Resource Throughput Greedy (CR-Throughput).** CR-Throughput is a communication-resource greedy baseline adapted from Ronen and Ben-Moshe [10]. At each time slot, each ground station selects the currently serviceable task with the largest immediately deliverable data volume.
- **Earliest Deadline First (EDF).** EDF is a bid-agnostic scheduling baseline that processes tasks according to the earliest-deadline-first rule. Unlike STAR-GS, it does not use bids for admission decisions and prioritizes tasks only by deadline urgency.
- **Highest-Bid-First with Naive Assignment (HBF-NA).** HBF-NA is a bid-greedy baseline inspired by auction-based resource allocation [21]. It admits tasks in descending bid order and assigns each task to the first serviceable ground station, without best-fit or slack-based feasibility comparison.
- **Satellite-Centric Best-Visibility (SC-BV).** SC-BV is a satellite-centric baseline inspired by LEO ground-station selection studies [9]. Each satellite selects the ground station with the longest total visibility duration, through which all its tasks are scheduled.

C. Overall Performance

In Fig. 2, we evaluate STAR-GS under varying system and workload settings, including task count, ground station bandwidth, deadline slack, task demand, bid dispersion, and task acceptance ratio. Unless otherwise specified, each data point is averaged over 10 runs.

Fig. 2(a) compares GSaaS revenue as the number of tasks increases from 100 to 500. STAR-GS consistently achieves the highest revenue, and its advantage becomes more pronounced under heavier workloads. At 500 tasks, EDF reaches about 76.8% of STAR-GS revenue, while HBF-NA and SC-BV achieve only about 51.0% and 22.4%, respectively. This shows that deadline-aware scheduling alone is not sufficient for revenue maximization, and that naive GS assignment or fixed satellite-centric station selection can lead to inefficient

resource usage. Overall, STAR-GS scales better by jointly considering task value, scheduling feasibility, and GS assignment.

Fig. 2(b) shows the impact of ground station bandwidth. When bandwidth is extremely limited, most methods obtain little or no revenue because few tasks are schedulable. As bandwidth increases, all methods improve, but STAR-GS benefits more from the additional capacity and remains the best-performing method. EDF is the closest baseline in the medium- and high-bandwidth regimes, but it still lags behind because it prioritizes urgent tasks rather than high-value tasks. This confirms that additional bandwidth is more effectively utilized when admission and assignment are both value-aware and feasibility-aware.

Fig. 2(c) evaluates the impact of deadline slack. Larger slack windows increase revenue for all methods because tasks have more feasible contact opportunities and are easier to schedule. STAR-GS consistently outperforms the baselines across different slack settings, with EDF again being the closest competitor. The gap indicates that relaxed deadlines alone do not eliminate the need for bid-aware admission, since the provider still needs to decide which feasible tasks are most valuable to serve.

Fig. 2(d) shows the impact of task demand. Revenue first increases as task demand grows from low to moderate levels, because each admitted task contributes more potential revenue while remaining schedulable. When the demand becomes too large, however, the feasible task set shrinks quickly and revenue drops for most methods. In this high-demand regime, CR-Throughput becomes more competitive because its deliverability-oriented rule favors tasks that can still be completed under tight resource constraints.

Fig. 2(e) shows the impact of bid dispersion. As the bid range becomes wider, all methods obtain higher revenue, but STAR-GS gains the most because it directly exploits bid heterogeneity during admission. The increasing gap between STAR-GS and bid-agnostic or throughput-oriented baselines confirms the benefit of value-aware task selection when task values differ significantly.

Fig. 2(f) compares the task acceptance ratio as the task volume increases. All methods admit a smaller fraction of tasks under heavier workloads due to stronger competition for limited bandwidth and visibility windows. STAR-GS maintains an acceptance ratio close to EDF while achieving much higher revenue. This suggests that STAR-GS does not improve revenue simply by rejecting more tasks; instead, it selects feasible tasks with higher value more effectively.

We further compare STAR-GS with TSN-MILP to evaluate solution quality and computational efficiency against an optimization benchmark. Since TSN-MILP has much higher computational cost and cannot scale to the larger settings used in Fig. 2, we evaluate it separately on smaller instances with task numbers from 30 to 270. TSN-MILP jointly optimizes task admission, ground station assignment, and bandwidth allocation using a mixed-integer formulation.

Fig. 3(a) compares the GSaaS revenue achieved by STAR-GS and TSN-MILP under different numbers of tasks. The

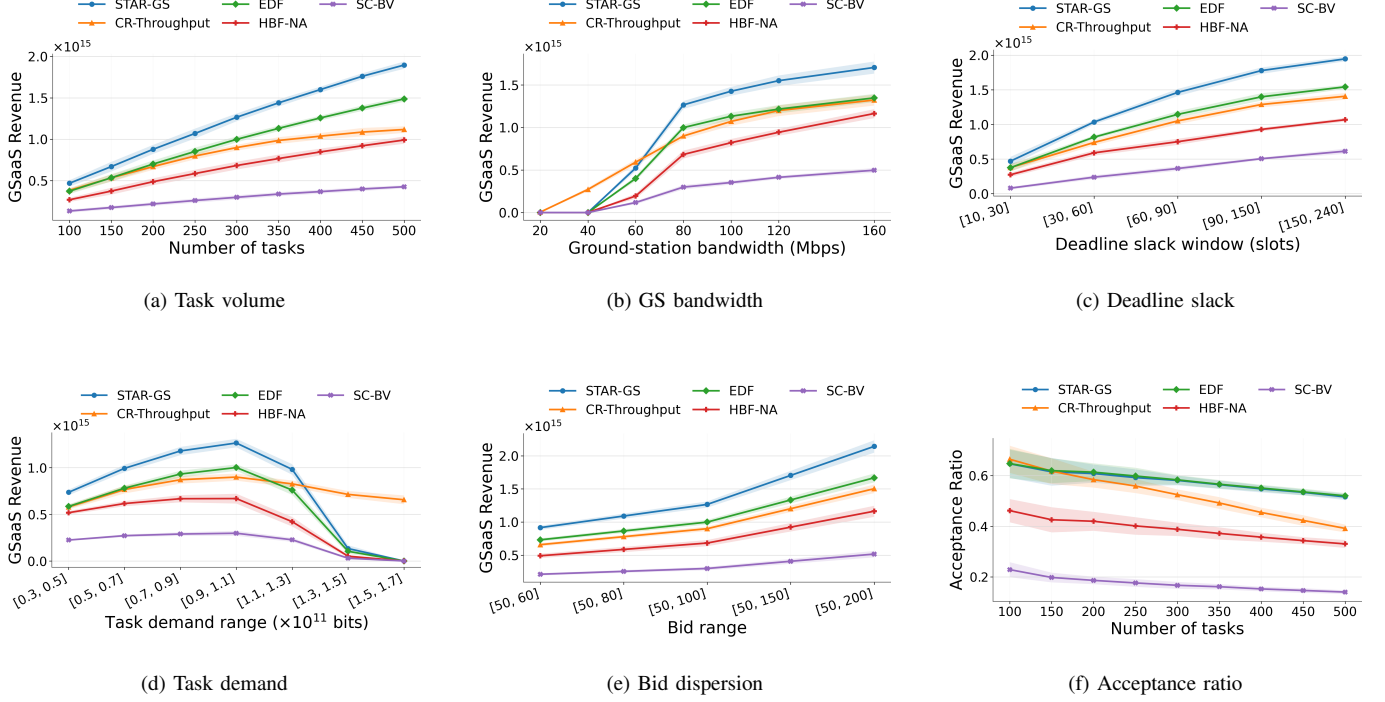


Fig. 2. Performance comparison under different system settings. Subfigures (a)–(e) report GSaaS revenue under different system parameters, while subfigure (f) reports the task acceptance ratio as the task volume increases.

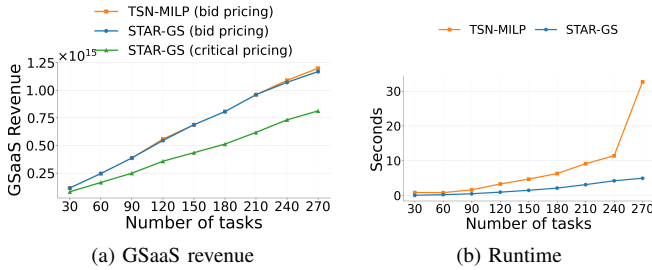


Fig. 3. Comparison between STAR-GS and TSN-MILP under different task numbers. Subfigure (a) reports GSaaS revenue, while subfigure (b) reports runtime.

TSN-MILP and STAR-GS curves with bid pricing report the revenue calculated from submitted bids, while the STAR-GS curve with critical pricing reports the truthful payment computed by the proposed truthful mechanism. As the number of tasks increases from 30 to 270, the revenue of all methods increases steadily. Under bid pricing, STAR-GS achieves revenue very close to TSN-MILP, indicating that STAR-GS can closely approximate the MILP solution in task admission and resource allocation. In contrast, STAR-GS with critical pricing obtains lower revenue because the critical price is the payment required to guarantee truthfulness, rather than the submitted bid.

Fig. 3(b) reports the runtime comparison. TSN-MILP shows a much faster increase in runtime as the task number grows, rising from less than 1 second at 30 tasks to over 32

seconds at 270 tasks. In contrast, STAR-GS increases more smoothly, from about 0.05 seconds to about 5 seconds over the same range. Therefore, STAR-GS achieves near-MILP revenue while requiring substantially less computation time, making it more suitable for larger-scale GSaaS scheduling.

VI. CONCLUSION

This paper presented STAR-GS, a truthful and visibility-aware scheduling mechanism for revenue-driven GSaaS resource allocation. By combining bid-aware admission, best-fit ground station assignment, EDF-based schedulability checking, and critical payment pricing, STAR-GS jointly addresses feasibility and strategic bidding under dynamic satellite-ground visibility constraints. Experiments with STK-generated contact windows show that STAR-GS consistently improves provider revenue over heuristic baselines; for example, under 500 tasks, the closest baseline achieves only about 76.8% of STAR-GS revenue, while naive bid-based and satellite-centric baselines achieve substantially less. STAR-GS also obtains revenue close to the MILP benchmark while reducing runtime significantly, e.g., about 5 seconds versus over 32 seconds at 270 tasks. These results suggest that provider-side GSaaS scheduling benefits from treating pricing and deadline-aware feasibility as coupled decisions. **Future work will extend STAR-GS to online request arrivals, time-varying channel conditions, dynamic ground-station capacity, adaptive reserve pricing, and multi-ground-station task splitting.**

REFERENCES

- [1] M. Hui, S. Zhai, D. Wang *et al.*, “A review of leo-satellite communication payloads for integrated communication, navigation, and remote sensing: Opportunities, challenges, future directions,” *IEEE Internet of Things Journal*, vol. 12, no. 12, pp. 18 954–18 992, 2025.
- [2] M. Centenaro, C. E. Costa, F. Granelli *et al.*, “A survey on technologies, standards and open challenges in satellite iot,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1693–1720, 2021.
- [3] F. S. Prol, R. M. Ferre, Z. Saleem *et al.*, “Position, navigation, and timing (pnt) through low earth orbit (leo) satellites: A survey on current status, challenges, and opportunities,” *IEEE Access*, vol. 10, pp. 83 971–84 002, 2022.
- [4] O. B. Osoro and E. J. Oughton, “A techno-economic framework for satellite networks applied to low earth orbit constellations: Assessing starlink, oneweb and kuiper,” *IEEE Access*, vol. 9, pp. 141 611–141 625, 2021.
- [5] C. Gu, Q. Wu, Z. Lai *et al.*, “Novaplan: An efficient plan of renting ground stations for emerging leo satellite networks,” in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.
- [6] I. Akyildiz, G. Morabito, and S. Palazzo, “Tcp-peach: a new congestion control scheme for satellite ip networks,” *IEEE/ACM Transactions on Networking*, vol. 9, no. 3, pp. 307–321, 2001.
- [7] W. Yang, L. Cai, S. Shu *et al.*, “Mobility-aware congestion control for multipath quic in integrated terrestrial satellite networks,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 11 620–11 634, 2024.
- [8] Y. C. Chou, L. Chen, H. Wang *et al.*, “Commercial dishes can be my ladder: Sustainable and collaborative data offloading in leo satellite networks,” in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.
- [9] D. Eddy, M. Ho, and M. J. Kochenderfer, “Optimal ground station selection for low-earth orbiting satellites,” in *2025 IEEE Aerospace Conference*, 2025, pp. 1–13.
- [10] R. Ronen and B. Ben-Moshe, “Maximizing nanosatellite throughput via dynamic scheduling and distributed ground stations,” *Sensors*, vol. 25, pp. 1–24, 12 2025.
- [11] Y. Ren, Q. Li, Y. Zhang *et al.*, “An evolutionary algorithm with memory guidance for data transmission scheduling optimization in communication satellite network,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 56, no. 2, pp. 1354–1368, 2026.
- [12] E. Filho, J. Silveira, and C. Marcon, “Blockchain applied in decentralization of ground stations to educational nanosatellites,” in *2023 IEEE 24th Latin American Test Symposium*, 2023, pp. 1–5.
- [13] N. Boschetti, C. Smethurst, G. Epiphaniou *et al.*, “Ground station as a service reference architectures and cyber security attack tree analysis,” in *2023 IEEE Aerospace Conference*, 2023, pp. 1–12.
- [14] G. Velusamy and R. Lent, “Ai-based ground station-as-a-service for optimal cost-latency satellite data downloading,” in *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*, 2022, pp. 2363–2368.
- [15] H. Zhao, S. Cen, and Y. Zhu, “The space above the sky: Uniting global-scale ground station as a service for efficient orbital data processing,” in *2024 IEEE 32nd International Conference on Network Protocols*, 2024, pp. 1–11.
- [16] G. R. Kim, D. Eddy, V. Srinivas *et al.*, “Scalable ground station selection for large leo constellations,” *arXiv preprint arXiv:2510.03438*, 2025.
- [17] Z. Sun, T. Ni, P. Hu *et al.*, “Spacesched: A constellation-wide scheduling system for resolving ground track congestion in remote sensing,” in *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*, 2025, p. 832–847.
- [18] J. Cheng, M. Fan, G. Yi *et al.*, “Uhts-drl: A deep reinforcement learning framework for integrated agile satellite observation and data transmission scheduling,” *Information Sciences*, vol. 740, pp. 1–19, 2026.
- [19] Y. Ren, Q. Li, Y. Guo *et al.*, “A distance similarity-based genetic optimization algorithm for satellite ground network planning considering feeding mode,” *Expert Systems with Applications*, vol. 268, pp. 1–17, 2025.
- [20] J. Yu, S. Liu, Y. Zou *et al.*, “Auction theory and game theory based pricing of edge computing resources: A survey,” *IEEE Internet of Things Journal*, vol. 12, no. 16, pp. 32 394–32 418, 2025.
- [21] X. Wang, X. Wang, C. Wang *et al.*, “Truthful online combinatorial auction-based mechanisms for task offloading in mobile edge computing,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 7, pp. 6488–6502, 2025.
- [22] D. Shah, D. N. C. Tse, and J. N. Tsitsiklis, “Hardness of low delay network scheduling,” *IEEE Transactions on Information Theory*, vol. 57, no. 12, pp. 7810–7817, 2011.
- [23] Planet Labs PBC, “Tasking API,” [Online]. Available: <https://docs.planet.com/develop/apis/tasking/>, accessed: May 21, 2026.
- [24] SkyFi, “Satellite Tasking Explained: How to Order Custom Satellite Imagery,” [Online]. Available: <https://skyfi.com/en/blog/satellite-tasking-explained>, accessed: May 21, 2026.
- [25] H. Myung and D. Goodman, “Single carrier fdma: A new air interface for long term evolution,” pp. 1–185, 10 2008.
- [26] Amazon Web Services, “AWS Ground Station User Guide: Work with Contacts,” [Online]. Available: <https://docs.aws.amazon.com/ground-station/latest/ug/contacts.html>, accessed: May 21, 2026.
- [27] W. S. Vickrey, “Counterspeculation, auctions, and competitive sealed tenders,” *Journal of Finance*, vol. 16, pp. 8–37, 1961.
- [28] C. L. Liu and J. W. Layland, *Scheduling algorithms for multiprogramming in a hard-real-time environment*. USA: Kluwer Academic Publishers, 2001, p. 179–194.
- [29] M. L. Dertouzos, “Control robotics: The procedural control of physical processes,” in *IFIP Congress*, 1974.
- [30] D. S. Johnson, A. Demers, J. D. Ullman *et al.*, “Worst-case performance bounds for simple one-dimensional packing algorithms,” *SIAM Journal on Computing*, vol. 3, no. 4, p. 299–325, Dec. 1974.
- [31] R. B. Myerson, “Optimal auction design,” *Mathematics of operations research*, vol. 6, no. 1, pp. 58–73, 1981.
- [32] T. Roughgarden, “Algorithmic game theory,” *Commun. ACM*, vol. 53, no. 7, pp. 78–86, 2010.
- [33] Analytical Graphics, Inc., “Systems Tool Kit (STK),” [Online]. Available: <https://www.agi.com/products/stk>, accessed: May 21, 2026.
- [34] MathWorks, “Constellation Modeling with the Orbit Propagator Block,” [Online]. Available: <https://www.mathworks.com/help/aeroblks/constellation-modeling-with-the-orbit-propagator-block.html>, accessed: May 22, 2026.
- [35] A. Y. Alhusenat, J. Tian, H. Rababah *et al.*, “Towards sustainable leo satellite operations: A degradation-aware scheduling framework for dod control and energy optimization,” *IEEE Open Journal of Vehicular Technology*, pp. 1–16, 2026.
- [36] S. Bhandari, T. X. Vu, and S. Chatzinotas, “Leo satellite-assisted task offloading for a near real-time earth observation service,” in *GLOBE-COM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 3033–3038.
- [37] H. Izakian, B. T. Ladani, K. Zamanifar *et al.*, “A continuous double auction method for resource allocation in computational grids,” in *2009 IEEE Symposium on Computational Intelligence in Scheduling*, 2009, pp. 29–35.
- [38] M.-W. Lee, S.-D. Yu, K. Kwon *et al.*, “Mixed-integer linear programming model for scheduling missions and communications of multiple satellites,” *Aerospace*, pp. 1–32, 2024.